

MAD & PWA Lab

Journal

Experiment No.	11
Experiment Title.	To use google Lighthouse PWA Analysis Tool to test the PWA functioning.
Roll No.	02
Name	Vanshika Ambwani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO6: Develop and Analyze PWA Features and deploy it over app hosting solution
Grade:	

EXPERIMENT NO: - 11

Name:- Vanshika, Gunajn, Akruti

Class:- D15A

Roll:No:- 2,6,11

AIM:- To use google Lighthouse PWA Analysis Tool to test the PWA functioning.

Theory: -

Google Lighthouse is an open-source tool designed to evaluate and optimize web applications based on multiple key parameters, including performance, accessibility, Progressive Web App (PWA) implementation, and best practices. It provides a detailed automated audit that helps developers improve their websites efficiently. Unlike traditional manual audits that can take days or weeks, Lighthouse generates insights within minutes with minimal setup.

One of Lighthouse's biggest advantages is its ease of use—simply run it on a webpage or provide a URL, and it will generate a comprehensive report. Lighthouse supports audits for both desktop and mobile versions of a website, ensuring an optimal user experience across different devices.

Key Features and Audit Metrics

1. Performance

This metric evaluates how efficiently a webpage loads and becomes interactive. It considers several factors, including:

- Page Load Speed – Measures how quickly content appears to users.
- First Contentful Paint (FCP) – Time taken for the first visible content to load.
- Largest Contentful Paint (LCP) – Measures how long the largest visible content takes to fully render.
- Cumulative Layout Shift (CLS) – Ensures content does not shift unexpectedly, improving user experience.
- Time to Interactive (TTI) – The time taken for the webpage to become fully functional.

Lighthouse assigns a performance score from 0 to 100, where:

- 100 → Top 2% of websites (98th percentile)
- 50 → Around the 75th percentile
- Lower scores → Indicate areas needing optimization

2. Progressive Web App (PWA) Score

With the increasing adoption of PWAs, web applications now strive to deliver an app-like experience. Lighthouse evaluates a website's PWA readiness based on Google's Baseline PWA Checklist, which includes:

- **Service Worker Implementation** – Enables offline functionality and background synchronization.
- **App Manifest Compliance** – Provides metadata for mobile app-like integration.
- **Viewport Configuration** – Ensures responsiveness across different screen sizes.
- **Performance in Script-Disabled Environments** – Verifies that the page functions properly even if JavaScript is disabled.

A high PWA score indicates that the application meets essential criteria for speed, reliability, and mobile usability.

3. Accessibility

This metric determines how well a website supports users with **visual, cognitive, or physical disabilities**. Lighthouse evaluates accessibility based on:

- **ARIA Attributes** – Enhances screen reader support (e.g., aria-required).
- **Text Alternatives for Media** – Ensures images, audio, and video content are accessible.
- **Semantic HTML Usage** – Encourages proper use of elements like <section>, <article>, and <button> for better screen-reader compatibility.

Unlike other metrics, **accessibility follows a pass/fail approach**—missing key features can **significantly impact the overall score**. Improving accessibility ensures **inclusivity for all users**.

4. Best Practices

Lighthouse also assesses whether the website follows **modern web development best practices**, including:

- **Use of HTTPS** – Ensures secure data transmission.
- **Avoiding Deprecated Code** – Prevents the use of outdated elements, directives, and libraries.
- **Secure Password Inputs** – Disables paste-into fields to reduce security risks.
- **User Security Alerts** – Provides notifications for **geo-location access and cookie usage**.

A high **Best Practices score** means the website meets industry standards, leading to better **security, maintainability, and overall performance**.

Manifest.json

```
{
```

```
"name": "Shopping App",
"short_name": "MyApp",
"description": "An online shopping platform with a wide range of products.",
"start_url": "/",
"scope": "/",
"display": "standalone",
"background_color": "#ffffff",
"theme_color": "#ff6600",
"icons": [
  {
    "src": "assets/images/banner-1.png",
    "sizes": "192x192",
    "type": "image/png",
    "purpose": "any maskable"
  },
  {
    "src": "assets/images/banner-2.png",
    "sizes": "512x512",
    "type": "image/png",
    "purpose": "any maskable"
  }
],
"shortcuts": [
  {
    "name": "Shop Now",
    "short_name": "Shop",
    "description": "Go directly to shopping",
    "url": "/index.html",
    "icons": [
      {
        "src": "assets/images/shortcut-icon.png",
```

```
    "sizes": "96x96",  
    "type": "image/png"  
  }  
]  
}  
]  
}
```

Output: -
AI For Mobile

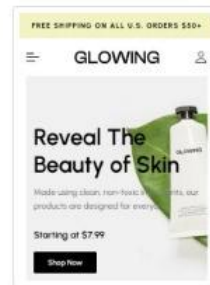
 https://gunjan-chandnani.github.io/pwa_deploy/



Performance

Values are estimated and may vary. The [performance score](#) is calculated directly from these metrics. [See calculator.](#)

▲ 0-49 ■ 50-89 ● 90-100



METRICS

[Expand view](#)

METRICS

[Expand view](#)

■ First Contentful Paint

2.2 s

▲ Largest Contentful Paint

4.6 s

● Total Blocking Time

0 ms

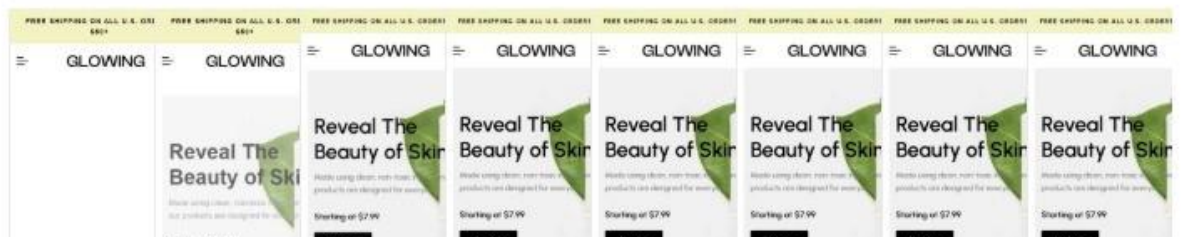
● Cumulative Layout Shift

0.028

● Speed Index

2.3 s

 [View Treemap](#)



DIAGNOSTICS

▲ Largest Contentful Paint element — 4,610 ms	▼
▲ Serve images in next-gen formats — Potential savings of 139 KiB	▼
▲ Enable text compression — Potential savings of 62 KiB	▼
▲ Use HTTP/2 — 16 requests not served via HTTP/2	▼
▲ Defer offscreen images — Potential savings of 36 KiB	▼
▲ Eliminate render-blocking resources — Potential savings of 170 ms	▼
■ Minify CSS — Potential savings of 6 KiB	▼
■ Serve static assets with an efficient cache policy — 10 resources found	▼
■ Avoid an excessive DOM size — 1,071 elements	▼



https://gunjan-chandnani.github.io/pwa_deploy/



Accessibility

These checks highlight opportunities to [improve the accessibility of your web app](#). Automatic detection can only detect a subset of issues and does not guarantee the accessibility of your web app, so [manual testing](#) is also encouraged.

ARIA

- ▲ Elements use prohibited ARIA attributes

These are opportunities to improve the usage of ARIA in your application which may enhance the experience for users of assistive technology, like a screen reader.

ADDITIONAL ITEMS TO MANUALLY CHECK (10)

Show

These items address areas which an automated testing tool cannot cover. Learn more in our guide on [conducting an accessibility](#)

ADDITIONAL ITEMS TO MANUALLY CHECK (10)

Show

These items address areas which an automated testing tool cannot cover. Learn more in our guide on [conducting an accessibility review](#).

PASSED AUDITS (24)

Show

NOT APPLICABLE (32)

Show



Best Practices

B|For Desktop

https://gunjan-chandnani.github.io/pwa_deploy/



Performance



Accessibility



Best Practices



SEO



Performance

Values are estimated and may vary. The [performance score is calculated](#) directly from these metrics. [See calculator](#).

▲ 0–49 ■ 50–89 ● 90–100



97

96

96

91

- First Contentful Paint

0.7 s

- Total Blocking Time

0 ms

- Speed Index

0.9 s

- Largest Contentful Paint

1.2 s

- Cumulative Layout Shift

0.001

[View Treemap](#)Show audits relevant to: [All](#) [FCP](#) [LCP](#) [TBT](#) [CLS](#)

